NEPAL COLLEGE OF INFORMATION TECHNOLOGY

Level: Bachelor • Programme: BE • Year: 2026

Course: .NET Technologies (Elective) • Full Marks: 100 • Time: 3 hrs

.NET Technologies Model Answers

Complete solution of the 2026 question paper

All questions including every **OR** alternative | Diagrams, tables & code examples

Covered: Q1–Q7 (17 answers in total)

Architecture • Page Life Cycle • Connection Strings • appsettings.json • EF Core

Exception Handling • LINQ • async/await • SOLID • State Management

Dependency Injection • RESTful Web APIs • Microservices • GC • Security • OOP

PREPARED BY

Arpan Adhikari

Contents

How to use this booklet. Every answer follows the same exam-friendly shape: **definition → labelled diagram → explanation of each part → table or code → advantages/conclusion**. Examiners award marks for (i) a correct definition, (ii) a neat labelled diagram, (iii) the technical keywords, and (iv) a short closing line. Reproduce the diagrams — they are quick to draw and carry easy marks.

Q1 (a). Illustrate the overall architecture of the .NET platform and explain the functions of its key components. 7 Marks

> **Definition.** The **.NET platform** is a software development framework from Microsoft that provides a **managed, language-independent and secure** environment for building and running applications — desktop, web, mobile and cloud. Code written in any .NET language is first compiled into a common **Intermediate Language (IL)**, which is then executed by the **Common Language Runtime (CLR)** sitting on top of the operating system.

The architecture is **layered**. Each layer only talks to the layer directly below it, which is what makes .NET language-independent and portable.

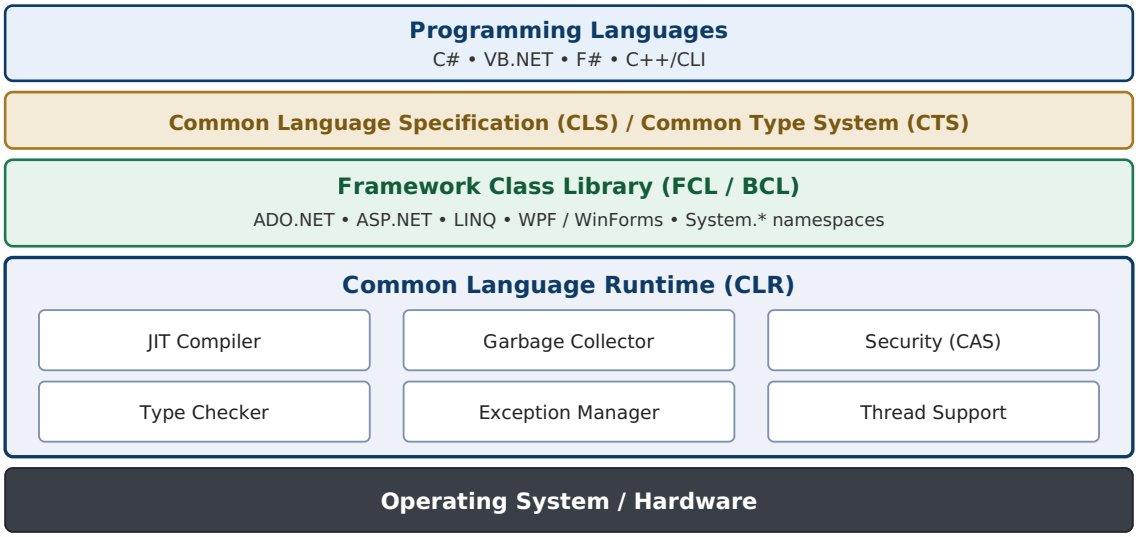

Fig 1.1: Layered architecture of the .NET platform

Functions of the Key Components

1. **Common Language Runtime (CLR)** — the execution engine (a virtual machine) of .NET. It loads assemblies, converts IL into native machine code using the **JIT compiler**, and provides run-time services: automatic memory management (**Garbage Collection**), **type safety**, **exception handling**, **thread management** and **security**. Code that runs under CLR supervision is called **managed code**.
2. **Common Type System (CTS)** — defines how types (value types and reference types) are declared and used, so objects written in different .NET languages can talk to each other. For example, `int` in C# and `Integer` in VB.NET both map to `System.Int32`.
3. **Common Language Specification (CLS)** — a subset of CTS rules that every .NET language must obey to guarantee **cross-language interoperability** (e.g. a class written in VB.NET can be inherited in C#).
4. **Framework Class Library (FCL) / Base Class Library (BCL)** — a huge collection of ready-made classes, interfaces and value types (collections, file I/O, networking, data access, XML, LINQ) organised into **namespaces** such as `System`, `System.IO` and `System.Data`. It saves developers from writing common code again and again.
5. **Assemblies and Metadata** — compiled code is packaged into assemblies (`.dll` / `.exe`) that contain IL code, **metadata** and a **manifest** (version, culture, referenced assemblies). Because metadata travels with the code, assemblies are **self-describing** and need no registry entries.

6. **Application Models** — the technologies built on top of the FCL that developers actually use: **ASP.NET / ASP.NET Core** (web), **ADO.NET / EF Core** (data), **WinForms / WPF** (desktop) and **.NET MAUI** (mobile).

Execution Flow (two-step compilation)

Fig 1.2: Two-step compilation — source → IL → native code

In words: the language compiler produces IL + metadata inside an assembly; at run time the CLR loads the assembly and the JIT compiler turns IL into native code for that particular CPU, just before each method runs. This two-step model is exactly what gives .NET its **language independence** and **platform portability**.

Advantages of this Architecture

- Language interoperability with one consistent programming model.
- Automatic memory management and type safety — far fewer leaks and crashes.
- Security through code verification and controlled (managed) execution.
- Simple deployment and versioning because assemblies are self-describing.

Q1 (b). Compare .NET Framework, .NET Core and the unified .NET platform in terms of runtime, tooling and application development.

8 Marks

.NET Framework (2002) is the original **Windows-only** implementation of .NET. **.NET Core** (2016) was a complete **cross-platform, open-source redesign** of it. From **.NET 5 (2020)** onwards Microsoft merged the two into a single platform simply called **.NET** (currently .NET 8 / 9), which is the future of the whole ecosystem.

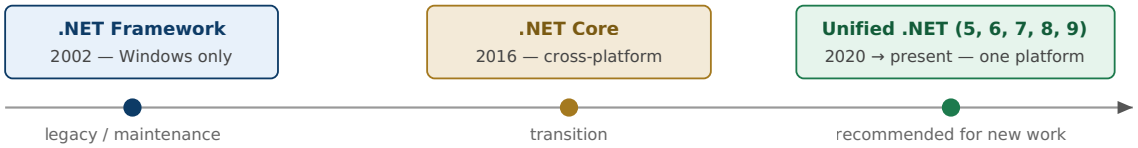

Fig 1.3: Evolution of the .NET platform

Comparison Table

Aspect	.NET Framework	.NET Core	Unified .NET (5+)
Release / status	2002; maintenance only (last 4.8.x)	2016; superseded by .NET 5	2020–present; actively developed (LTS every 2 years)
Platform support	Windows only	Windows, Linux, macOS	Windows, Linux, macOS + mobile (MAUI) + WebAssembly (Blazor)
Runtime	CLR tied to Windows; installed machine-wide in the GAC	CoreCLR — lightweight, modular, side-by-side versions, self-contained deployment	CoreCLR + Mono / NativeAOT; tiered JIT, ReadyToRun, AOT compilation
Library model	Monolithic FCL installed with the OS	CoreFX delivered as NuGet packages	Single unified BCL; .NET Standard no longer needed for new work
Tooling	Visual Studio + MSBuild; no cross-platform CLI	`dotnet` CLI, VS, VS Code; SDK-style `.csproj`	Same modern CLI/SDK plus hot reload, minimal APIs, container-first tooling
Application development	Web Forms, MVC 5, WCF, WPF, WinForms	ASP.NET Core (MVC + Web API unified), console; WPF/ WinForms from 3.0 (Windows only)	ASP.NET Core, Blazor, gRPC, Minimal APIs, MAUI, worker services, Azure Functions
Performance	Good but heavier	Very high (Kestrel server, `Span<T>`)	Best-in-class; improves with every release
Open source	Partially (reference source)	Fully open source (MIT)	Fully open source (MIT)
Not supported	—	Web Forms, WCF hosting, .NET Remoting	Web Forms, WCF server (use gRPC / CoreWCF)

1. Runtime Comparison

The .NET Framework CLR is installed **machine-wide**, so every application on that computer shares one runtime version — upgrading it can break older applications. **CoreCLR** (used by .NET Core and unified .NET) supports **side-by-side versioning** and **self-contained deployment**, where the runtime is shipped inside the application folder. The unified platform adds **tiered compilation**, **ReadyToRun** images and **Native AOT** for very fast startup and small containers.

2. Tooling Comparison

.NET Framework development is tied to **Visual Studio and MSBuild on Windows**. .NET Core introduced the cross-platform **dotnet CLI** (`dotnet new`, `build`, `run`, `publish`, `test`) and lightweight SDK-style project files, so development is possible in VS Code on any OS with easy CI/CD and Docker integration. Unified .NET carries all of this forward and adds hot reload and first-class container support.

3. Application Development Comparison

Legacy models (**Web Forms, WCF**) exist only on .NET Framework. .NET Core merged MVC and Web API into one **ASP.NET Core** model. Unified .NET widens this further with **Blazor** (C# running in the browser), **gRPC** for high-performance service calls, **Minimal APIs** and **.NET MAUI** for desktop and mobile from a single code base.

> **Conclusion:** .NET Framework is **legacy** — keep it only for existing Windows applications; .NET Core was the **transitional** cross-platform rewrite; **unified .NET (5+)** is the single, modern, high-performance platform recommended for all new development.

Q2 (a). Describe the sequence of events in the ASP.NET Page Life Cycle. Explain the purpose of the major stages involved in processing and rendering a web page. 7 Marks

> **Definition.** In ASP.NET Web Forms, every request for an `.aspx` page makes the runtime **create the page object, run it through a fixed sequence of stages and events, generate HTML, and then destroy the page**. This fixed sequence is called the **Page Life Cycle**. Knowing it is essential so that code is written in the correct event (for example, dynamic controls must be created in `PreInit`, but data binding is done in `Load`).

Stages of the Life Cycle

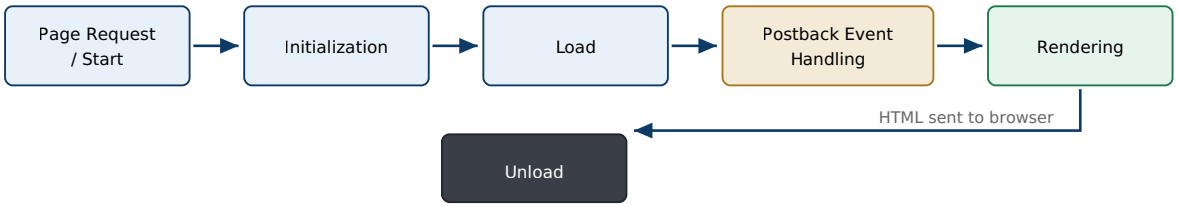

Fig 2.1: Stages of the ASP.NET Web Forms page life cycle

Purpose of the Major Stages

- **Page Request / Start:** ASP.NET decides whether the page must be parsed and compiled or served from cache, creates the `Page` object, and sets `Request`, `Response` and `IsPostBack`.
- **Initialization:** every control gets its unique ID and its properties; themes and master pages are applied. ViewState is **not** available yet.
- **Load:** control values are restored from ViewState and posted data; this is where databases are accessed and controls are data-bound.
- **Postback Event Handling:** the event raised by the user (button click, selection change) is executed, along with validation.
- **Rendering:** each control writes its HTML into the response stream, and ViewState is saved.
- **Unload:** the page has already been sent to the browser; resources are released and cleanup is done.

Sequence of Events (in order of execution)

#	Event	Purpose / Typical Use
1	**PreInit**	First event. Check `IsPostBack`; set master page and theme; create **dynamic controls**. ViewState is not yet loaded.
2	**Init**	Each control is initialised (children first, then the page). Control properties can be set; ViewState still unavailable.
3	**InitComplete**	Initialisation finished; **ViewState tracking is switched on** from here.
4	**PreLoad**	Fires after ViewState and postback data have been loaded into the controls, just before Load.
5	**Load** (`Page_Load`)	**Most used event.** Page and all controls are ready; connect to database, bind data, read control values. Use `if (!IsPostBack)` for one-time logic.
6	**Control / Postback events**	Server-side events caused by the user — `Button_Click`, `SelectedIndexChanged`; validation also runs here.
7	**LoadComplete**	All controls loaded and all postback events processed — good for logic that needs every control ready.
8	**PreRender**	**Last chance** to change page/control content before HTML is generated (final data binding).
9	**SaveStateComplete**	ViewState has been serialised into the hidden `__VIEWSTATE` field; later changes to controls will not persist.
10	**Render**	Not an event but a **method** — every control writes its own HTML markup to the output stream.
11	**Unload**	Page already sent to the client; close files and DB connections, free resources. The response can no longer be modified.

```
// Typical use of the Load event
protected void Page_Load(object sender, EventArgs e)
{
    if (!IsPostBack)          // runs only on the first GET, not on postbacks
    {
        GridView1.DataSource = GetStudents();
        GridView1.DataBind();
    }
}
```

Postback note: on a postback the same cycle runs again, but ViewState and the posted form data restore the control values **between Init and Load**, and the event of the control that caused the postback (e.g. `Click`) fires **after Load**. This is why `if (!IsPostBack)` is needed — otherwise the data binding would overwrite whatever the user typed.

Q2 (b). Define a connection string in a .NET application. Explain the important parameters commonly included in a SQL Server connection string and state their purposes.

8 Marks

> **Definition.** A **connection string** is a single line of text made up of **semicolon-separated `key = value` pairs** that tells a data provider (such as ADO.NET's `SqlConnection` or EF Core) everything it needs in order to **locate and open a database connection** — which server, which database, which credentials, and how the connection should behave (timeout, encryption, pooling).

A connection string is normally kept **outside the source code** — in `web.config` / `app.config` (`<connectionStrings>`) for classic ASP.NET, or in **`appsettings.json`** under the `"ConnectionStrings"` section for ASP.NET Core — so the same build can be pointed at a development, testing or production database without recompiling.

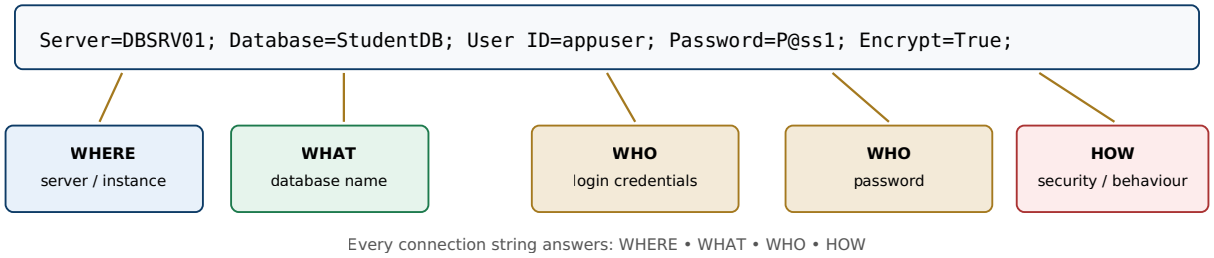

Fig 2.2: Anatomy of a SQL Server connection string

```
// appsettings.json (ASP.NET Core)
"ConnectionStrings": {
  "Default": "Server=DBSRV01;Database=StudentDB;User ID=appuser;Password=P@ss1;
              Encrypt=True;TrustServerCertificate=False;Connect Timeout=30;"
}

// using it with ADO.NET
using var con = new SqlConnection(config.GetConnectionString("Default"));
con.Open();
```

Important Parameters of a SQL Server Connection String

Parameter (aliases)	Purpose	Example
Server (Data Source, Addr)	Name or IP of the SQL Server machine/ instance to connect to — optionally with instance name or port.	`Server=DBSRV01\SQLEXPRESS`, `Server=10.0.0.5,1433`, `.`
Database (Initial Catalog)	The default database to open on that server.	`Database=StudentDB`
Integrated Security (Trusted_Connection)	`True` / `SSPI` means use the current **Windows account** (no username/ password in the string); `False` means SQL authentication.	`Integrated Security=SSPI`
User ID (uid)	The SQL Server login name, used when SQL authentication is chosen.	`User ID=appuser`
Password (pwd)	Password of that SQL login. Must be protected (user secrets, environment variables, Key Vault) — never hard-coded.	`Password=P@ss1`
Connect Timeout (Connection Timeout)	Seconds to wait while trying to establish the connection before throwing an error (default 15).	`Connect Timeout=30`
Encrypt	Whether TLS/SSL encryption is used for data travelling between application and server.	`Encrypt=True`
TrustServerCertificate	If `True`, the server's TLS certificate is not validated — acceptable only in development.	`TrustServerCertificate=False`
Pooling / Min-Max Pool Size	Turns on **connection pooling** (on by default) and sets its limits. Reusing open connections greatly improves performance.	`Pooling=True;Max Pool Size=100`
MultipleActiveResultSets (MARS)	Allows more than one open reader/ command on the same connection at the same time.	`MultipleActiveResultSets=True`
Application Name	Identifies your application inside SQL Server monitoring and profiling tools.	`Application Name=StudentApp`
AttachDbFilename	Attaches an `.mdf` database file directly (used with LocalDB during development).	`AttachDbFilename=|DataDirectory|\db.mdf`

Best Practices

- Keep connection strings in **configuration** (appsettings.json, user secrets, environment variables, Azure Key Vault) — never inside compiled source code or a public repository.
- Prefer **Integrated Security** or managed identities over an embedded password wherever possible.
- Always **dispose** connections with a `using` block so they go back to the pool immediately.
- Use `SqlConnectionStringBuilder` when building a string from user input — it escapes values safely and prevents connection-string injection.

Q3 (a). Explain the role and structure of the appsettings.json file in ASP.NET Core and discuss its importance in configuration management. 7 Marks

Role. `appsettings.json` is the **main configuration file of an ASP.NET Core application**. It replaces the XML `web.config` of classic ASP.NET and stores settings such as **connection strings, logging levels, feature flags, API keys and custom application options** in readable JSON. At startup the host loads it automatically into the unified **`IConfiguration`** system, from where any class can read it through dependency injection.

Structure of the File

The file is **hierarchical**: nested JSON objects form **sections**, and any value is addressed by a **colon-separated path** such as `"AppSettings:Smtp:Host"`.

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "ConnectionStrings": {
    "Default": "Server=.;Database=StudentDB;Trusted_Connection=True;"
  },
  "AllowedHosts": "*",
  "AppSettings": {                        // custom section
    "AppName": "Student Portal",
    "MaxPageSize": 50,
    "Smtp": { "Host": "smtp.ncit.edu.np", "Port": 587 }
  }
}
```

- **Logging** — how much detail each category writes to the log.
- **ConnectionStrings** — read conveniently with `GetConnectionString("Default")`.
- **AllowedHosts** — host filtering for security.
- **Custom sections** — any application-specific settings you define yourself.

Reading the Values

```
// (a) Direct access through IConfiguration (injected)
public HomeController(IConfiguration config)
{
    string name = config["AppSettings:AppName"];
    string cs   = config.GetConnectionString("Default");
}

// (b) Strongly-typed Options pattern  – recommended
public class AppSettings { public string AppName { get; set; } public int MaxPageSize { get;
set; } }

builder.Services.Configure<AppSettings>(builder.Configuration.GetSection("AppSettings"));

public class ReportService
{
    public ReportService(IOptions<AppSettings> opt)
        => Console.WriteLine(opt.Value.AppName);   // "Student Portal"
}
```

The Layered Configuration Model

`appsettings.json` is only **one of several configuration providers**. They are loaded in a fixed order and **later sources override earlier ones**, so secrets never have to be committed to source control.

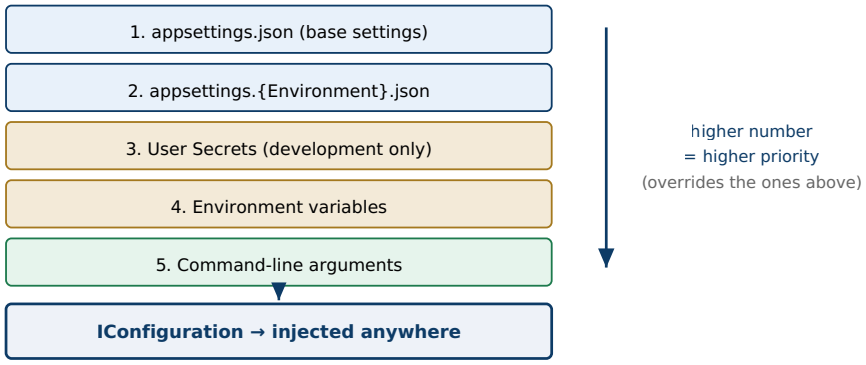

Fig 3.1: Layered configuration providers merged into one IConfiguration

Importance in Configuration Management

1. **Environment-specific overrides:** files such as `appsettings.Development.json` and `appsettings.Production.json` are layered on top of the base file according to the `ASPNETCORE_ENVIRONMENT` variable. Each environment overrides only the values that differ — **no code change and no recompilation**.
2. **Separation of configuration from code:** changing a database server, an SMTP host or a log level is an edit to a text file, not a new build and deployment.
3. **Security:** because environment variables and user secrets sit **above** the JSON file in priority, real passwords stay out of source control while the file keeps harmless placeholders.
4. **Strong typing and validation:** the **Options pattern** binds a section to a POCO class, giving IntelliSense, compile-time safety and `ValidateDataAnnotations()` checks at startup instead of run-time typos.
5. **Change reload:** with `reloadOnChange: true` (the default) together with `IOptionsSnapshot` / `IOptionsMonitor`, an edited setting is picked up **without restarting** the application.
6. **Centralisation + DI integration:** all settings live in one predictable place and flow to any class through the DI container, keeping configuration concerns cleanly separate from business logic.

> **One-line conclusion:** `appsettings.json` makes an ASP.NET Core application **configurable rather than hard-coded** — the same compiled application can run correctly in development, testing and production simply by changing layered configuration values.

Q3 (b). Discuss the role and major advantages of Entity Framework Core in performing database access operations.

8 Marks

Definition. Entity Framework Core (EF Core) is Microsoft's modern, lightweight, cross-platform **Object–Relational Mapper (ORM)** for .NET. Its role is to remove the "impedance mismatch" between **object-oriented C# code** and **relational tables**: the developer works with entity classes and LINQ queries, and EF Core generates the SQL, executes it, and converts the returned rows back into objects.

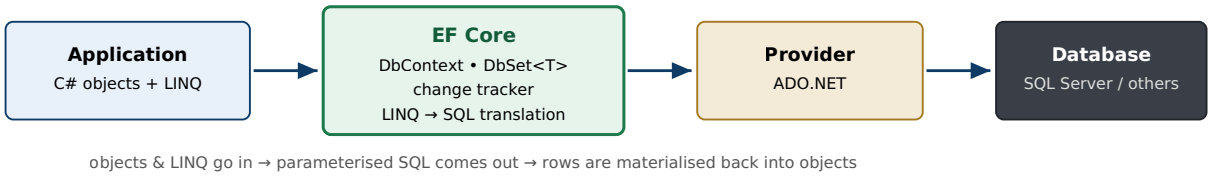

Fig 3.2: EF Core sits between application objects and the relational database

Core Responsibilities (its role)

- **Object–relational mapping:** maps classes ↔ tables, properties ↔ columns, and relationships (1:1, 1:N, N:N) using conventions, data annotations or the Fluent API.
- **Query translation:** converts LINQ expressions into provider-specific, **parameterised SQL**, and turns the result rows into entity objects.
- **Change tracking and unit of work:** the `DbContext` remembers the state of every loaded entity (*Added, Unchanged, Modified, Deleted*) and writes all pending changes in **one transaction** when `SaveChanges()` is called.
- **Schema management (Migrations):** `Add-Migration` / `Update-Database` evolve the database schema from the model and keep it version-controlled together with the code (Code-First); Database-First scaffolding is also supported.

A Complete Example (setup + CRUD)

```
public class Student                        // entity → table "Students"
{
    public int Id { get; set; }             // primary key by convention
    public string Name { get; set; }
    public double GPA { get; set; }
}

public class SchoolContext : DbContext
{
    public SchoolContext(DbContextOptions<SchoolContext> o) : base(o) { }
    public DbSet<Student> Students { get; set; }
}

// Program.cs — registration
builder.Services.AddDbContext<SchoolContext>(o =>
    o.UseSqlServer(builder.Configuration.GetConnectionString("Default")));

// CRUD
db.Students.Add(new Student { Name = "Arpan", GPA = 3.9 });   // CREATE → INSERT
var toppers = db.Students.Where(s => s.GPA >= 3.5)
                         .OrderByDescending(s => s.GPA).ToList();  // READ → SELECT
var s1 = db.Students.Find(1); s1.GPA = 4.0;                    // UPDATE → UPDATE
db.Students.Remove(db.Students.Find(2));                       // DELETE → DELETE
db.SaveChanges();                           // all changes committed in one transaction
```

Major Advantages of EF Core

1. **Higher productivity, far less boilerplate:** no manual `SqlConnection` / `SqlCommand` / `SqlDataReader` code and no hand-written SQL for routine work — CRUD becomes a few lines of C#.
2. **Strong typing and compile-time checking:** LINQ queries are ordinary C#, so wrong column names and type mismatches are caught by the compiler, with full IntelliSense — mistakes that a SQL string would hide until run time.
3. **Database independence:** providers exist for SQL Server, SQLite, PostgreSQL, MySQL, Oracle, Cosmos DB and InMemory — switching database mostly means changing the provider and connection string.
4. **Security by default:** every generated query is **parameterised**, which protects the application from **SQL injection** automatically.
5. **Migrations:** safe, repeatable, versioned schema evolution kept in sync with the entity model across development, test and production.
6. **Performance features:** compiled queries, no-tracking queries (`AsNoTracking()`), control over eager / lazy / explicit loading, statement batching, and full async support (`ToListAsync`, `SaveChangesAsync`) for scalable web applications.
7. **Testability and maintainability:** the InMemory and SQLite providers allow fast unit tests without a real database; centralised mapping keeps data access consistent and easy to refactor, and it integrates naturally with DI and the Repository pattern.
8. **Cross-platform and open source:** runs on Windows, Linux and macOS and is developed openly on GitHub.

> **Limitation (write this for balance):** for extremely performance-critical or very complex queries, EF Core may generate sub-optimal SQL. In such cases developers can still drop down to raw SQL with `FromSqlInterpolated()` or call stored procedures — EF Core allows both approaches to be mixed in the same application.

Q4 (a). Explain the exception-handling mechanism in C#. Discuss the functions of the try, catch and finally blocks with an appropriate example.

7 Marks

> **Definition.** An **exception** is an object derived from `System.Exception` that the CLR throws when an abnormal condition occurs at run time (divide by zero, null reference, file not found, invalid input). **Structured Exception Handling (SEH)** in C# is the mechanism of detecting such errors with **try–catch–finally** blocks and responding to them in a controlled way, instead of letting the program crash.

Functions of the Blocks

- `try` — encloses the "guarded" code that **might** throw an exception. It must be followed by at least one `catch` block or a `finally` block.
- `catch` — **handles** one specific type of exception. Several `catch` blocks may be written; they are tested from top to bottom and the **first matching** one runs. An exception filter (`when`) can make catching conditional.
- `finally` — **always executes**, whether an exception occurred or not, and even if the `try` or `catch` block contains a `return`. It is used for **cleanup**: closing files, closing database connections, releasing locks.
- `throw` — raises an exception. A bare `throw;` inside a `catch` re-throws the current exception while **preserving the original stack trace**.

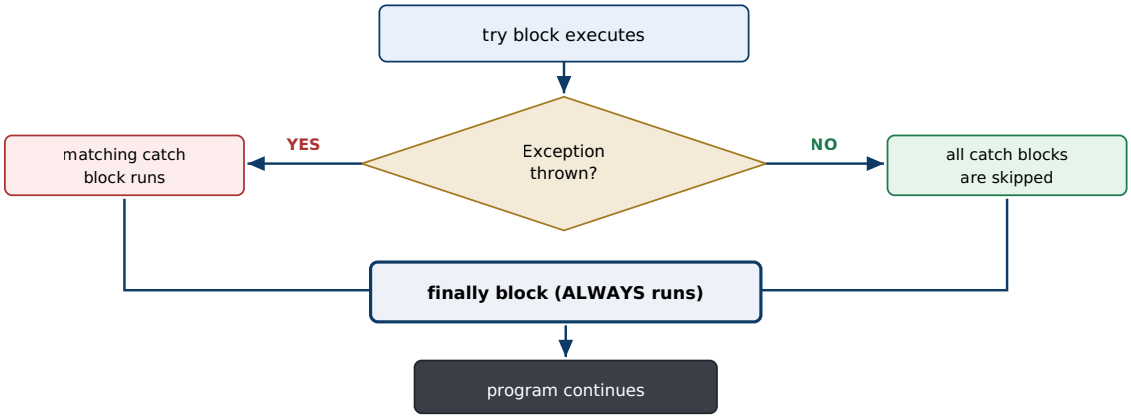

Fig 4.1: Control flow of try–catch–finally

Example

```
try
{
    int[] marks = { 90, 85, 78 };
    Console.Write("Enter index and divisor: ");
    int idx = int.Parse(Console.ReadLine());
    int div = int.Parse(Console.ReadLine());
    Console.WriteLine(marks[idx] / div);          // may throw several exceptions
}
catch (DivideByZeroException ex)                  // most specific first
{
    Console.WriteLine("Error: cannot divide by zero. " + ex.Message);
}
catch (IndexOutOfRangeException ex)
{
    Console.WriteLine("Error: invalid index. " + ex.Message);
}
catch (FormatException) when (!Console.IsInputRedirected)   // exception filter
{
    Console.WriteLine("Error: please enter numbers only.");
}
catch (Exception ex)                              // general handler LAST
{
    Console.WriteLine("Unexpected error: " + ex.Message);
    throw;                                        // re-throw, keeps stack trace
}
finally
{
    Console.WriteLine("Cleanup: closing resources...");     // ALWAYS runs
}
```

Important Rules and Points

1. **Order matters:** catch blocks must go from **most specific to most general**. Writing `catch (Exception)` before a derived type is a **compile-time error**, because the general handler would swallow everything.
2. `finally` executes even if `return` or `break` appears inside `try`/`catch` — this is what guarantees cleanup. The `using` statement is simply syntactic sugar over a `try/finally` that calls `Dispose()`.
3. If **no catch matches**, the exception **propagates up the call stack**; if it is never handled, the CLR terminates the program (after running the `finally` blocks).
4. Useful members of an exception object: `Message`, `StackTrace`, `InnerException`, `Source`.
5. **Custom exceptions** are made by deriving from `Exception`, e.g. `class InvalidGpaException : Exception { }`, for domain-specific errors.
6. Do not use exceptions for normal control flow — they are expensive; prefer validation or `TryParse` where possible.

> **Benefits:** separates error-handling code from normal logic, prevents crashes, guarantees resource cleanup, preserves diagnostic information (stack trace), and allows graceful recovery — making applications robust and maintainable.

Q4 (b). Define Language Integrated Query (LINQ). Explain how LINQ enables querying and manipulation of data from different data sources within .NET applications. 8 Marks

Definition. LINQ (Language Integrated Query), introduced in .NET 3.5 / C# 3.0, brings **query capability directly into the C# and VB.NET languages**. With one consistent, SQL-like syntax the developer can query **many different data sources** — in-memory collections, relational databases, XML documents and DataSets — instead of learning a separate query language for each one. Queries are built from **standard query operators** (`Where, Select, OrderBy, GroupBy, Join, Sum, Count` ...) which are extension methods on `IEnumerable<T>` and `IQueryable<T>`.

One Query Model, Many Data Sources

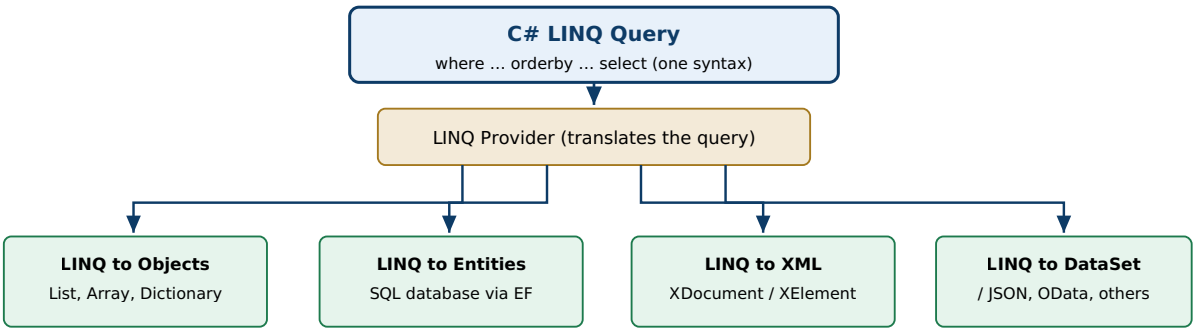

Fig 4.2: The same LINQ query works over any data source through its provider

The Two Syntaxes

```
var students = new List<Student> {
    new("Arpan","CS",3.9), new("Bina","Math",3.2), new("Chandan","CS",3.6) };

// (a) Query syntax — SQL-like, easy to read
var q1 = from s in students
         where s.Course == "CS" && s.GPA >= 3.5
         orderby s.GPA descending
         select new { s.Name, s.GPA };

// (b) Method syntax — lambda based, exactly equivalent
var q2 = students.Where(s => s.Course == "CS" && s.GPA >= 3.5)
                 .OrderByDescending(s => s.GPA)
                 .Select(s => new { s.Name, s.GPA });

// Manipulation: grouping and aggregation
var byCourse = students.GroupBy(s => s.Course)
                       .Select(g => new { Course = g.Key,
                                          Count  = g.Count(),
                                          AvgGpa = g.Average(s => s.GPA) });

// Same syntax against a database (EF Core) — translated into SQL
var toppers = await db.Students.Where(s => s.GPA >= 3.5).ToListAsync();
```

Common Standard Query Operators

Category	Operators	Purpose
Filtering	`Where`, `OfType`	Select only the rows/objects that satisfy a condition
Projection	`Select`, `SelectMany`	Shape the output — pick columns or build new/anonymous objects
Ordering	`OrderBy`, `ThenBy`, `Reverse`	Sort the result
Grouping / joining	`GroupBy`, `Join`, `GroupJoin`	Group related items, or combine two sources
Aggregation	`Count`, `Sum`, `Min`, `Max`, `Average`	Produce a single summary value
Element / set	`First`, `Single`, `Distinct`, `Union`, `Any`, `All`	Take one element, or perform set operations
Conversion	`ToList`, `ToArray`, `ToDictionary`	Force execution and materialise the result

How LINQ Enables Querying and Manipulation of Different Data Sources

1. **Unified query model:** one syntax queries objects, SQL databases, XML and more. Only the **provider** changes underneath — the developer's code looks the same.
2. **Provider translation (`IQueryable`):** against a database, the LINQ query is built as an **expression tree** which the provider translates into efficient, **parameterised SQL executed on the server** — so only the required rows travel over the network, and SQL injection is prevented. For in-memory collections (`IEnumerable`) the operators simply run as optimised loops.
3. **Compile-time checking and IntelliSense:** a query is ordinary C# code, so a misspelled column name or a wrong type is caught by the compiler — errors that a SQL string would reveal only at run time. Full IDE support for auto-completion, refactoring and debugging.
4. **Strong typing:** results are typed objects (including anonymous types), so there is no casting from `DataReader` columns and fewer run-time errors.
5. **Declarative and readable:** the developer states **what** data is required, not **how** to loop and filter — several nested `foreach`/`if` blocks shrink into one self-documenting query.
6. **Deferred execution:** a query is only an expression until it is enumerated (`foreach`, `ToList()`, `Count()`). This lets queries be **composed in steps** (add paging or filters conditionally) and always run against the latest data; immediate execution is available with `ToList()`/`ToArray()`.
7. **Powerful composability:** filtering, projection, ordering, grouping, joining, set operations and aggregation can be chained fluently; **PLINQ** (`AsParallel()`) parallelises in-memory queries over multiple cores.
8. **Less boilerplate, fewer bugs:** common data-manipulation patterns become one-liners, which raises productivity and maintainability.

> **Conclusion:** LINQ turns data access from error-prone, string-based or loop-based code into **first-class, type-safe language constructs**, giving one consistent, readable and safe way to query and manipulate every data source in a .NET application.

Q5 (a). Discuss asynchronous programming in .NET with reference to the async and await keywords and explain its benefits. 7 Marks

Concept. Asynchronous programming lets a program **start a long-running operation** (file or network I/O, database query, web request) and **continue doing other work** instead of blocking the current thread while waiting for it to finish. .NET implements this through the **Task-based Asynchronous Pattern (TAP)**: an asynchronous method returns a `Task` or `Task<T>` — a "promise" of a future result — and the **`async`** and **`await`** keywords (C# 5.0) let this be written in ordinary sequential-looking code.

The async and await Keywords

- **`async`** — a **method modifier** that (i) allows `await` to be used inside the method and (ii) tells the compiler to rewrite the method into a **state machine**. An async method returns `Task`, `Task<T>`, or `void` (`void` only for event handlers).
- **`await`** — applied to a Task. If the task is already finished, execution simply continues. Otherwise the method **suspends**, **returns control to its caller and frees the thread**, and registers a **continuation** that resumes the rest of the method when the task completes. The result is automatically unwrapped (`T` from `Task<T>`) and exceptions propagate normally into `try/catch`.

Key idea in one line: `await` does **not** create a new thread — it **releases** the current thread while the I/O is being performed by the operating system.

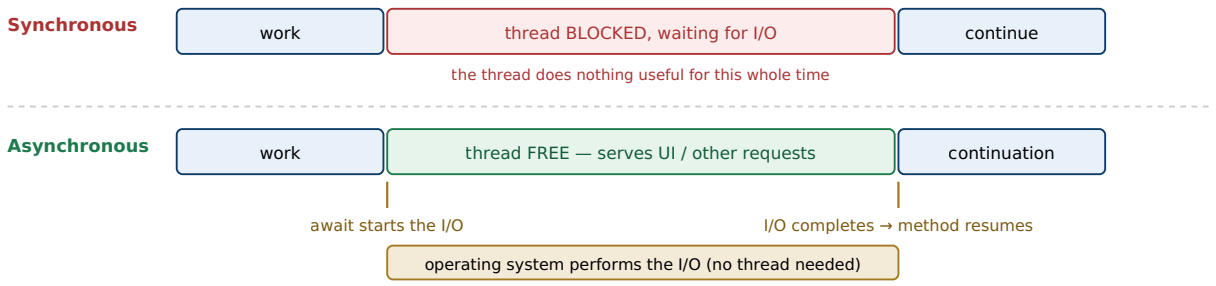

Fig 5.1: await frees the thread during I/O instead of blocking it

Example

```
// BAD — synchronous / blocking: UI freezes, server thread wasted
public string Download(string url)
{
    var http = new HttpClient();
    return http.GetStringAsync(url).Result;      // .Result BLOCKS the thread!
}

// GOOD — asynchronous version
public async Task<int> GetPageLengthAsync(string url)
{
    using var http = new HttpClient();
    Console.WriteLine("Requesting...");
    string html = await http.GetStringAsync(url);   // thread is released here
    Console.WriteLine("Received.");                 // continuation resumes here
    return html.Length;
}

// Calling code — overlapping work
public async Task RunAsync()
{
    Task<int> t = GetPageLengthAsync("https://example.com");  // starts the download
    DoOtherWork();                                            // runs meanwhile
    int length = await t;                                     // non-blocking wait
}

// ASP.NET Core action — scalable server code
[HttpGet]
public async Task<IEnumerable<Student>> Get() => await _db.Students.ToListAsync();
```

Benefits of async / await

1. **Responsive user interface (desktop / mobile):** the UI thread is released at every `await`, so the window keeps repainting and responding to clicks during a long operation — no "Not Responding" freeze. The continuation automatically resumes back on the UI thread (SynchronizationContext).
2. **Server scalability (ASP.NET Core):** while a request waits for a database or HTTP call, its thread returns to the **thread pool** and serves other requests. The same server therefore handles **far more concurrent users with fewer threads** — higher throughput with no extra hardware.
3. **Efficient use of resources:** each thread costs about 1 MB of stack memory; async I/O uses operating-system completion callbacks instead of parked threads, cutting memory use and context switching.
4. **Natural concurrency:** independent operations can be started together and awaited as a group — `await Task.WhenAll(t1, t2, t3)` — so the total time is that of the slowest one, not the sum.
5. **Readable and maintainable code:** compared with old callback / APM styles, `async/await` keeps the sequential structure and works normally with `try/catch/finally`, loops and `using` — asynchronous code that reads like synchronous code.
6. **Cancellation and progress support:** a `CancellationToken` can cancel a long operation cleanly, and `IProgress<T>` can report progress to the UI.

> **Best practices to mention:** follow "**async all the way**" — never call `.Result` or `.Wait()` (they block and can cause **deadlock**); suffix asynchronous method names with **Async**; use `Task.Run` only for **CPU-bound** work, not for I/O; and always accept a `CancellationToken` in long-running operations.

Q5 (b). Explain the SOLID principles and discuss their importance in developing maintainable and extensible software applications.

8 Marks

SOLID is an acronym for five object-oriented design principles (popularised by Robert C. Martin) that guide developers towards software that is **easy to understand, extend, test and maintain**. They are the foundation of good .NET architecture — ASP.NET Core itself (dependency injection, the middleware pipeline, small interfaces) is built around them.

Fig 5.2: The five SOLID principles

The Five Principles

Principle	Statement (what it means)	Example in .NET
S — Single Responsibility (SRP)	A class should have **only one reason to change** — that is, only one job.	Split `ReportService` (calculation) from `ReportPrinter` (output) and `ReportRepository` (saving) instead of one class doing all three.
O — Open / Closed (OCP)	Software entities should be **open for extension but closed for modification**.	A new payment type is added by writing a new `IPayment` implementation — no editing of existing, already-tested code. The middleware pipeline is extended without changing the framework.
L — Liskov Substitution (LSP)	A **subtype must be usable anywhere its base type is expected**, without breaking the program's behaviour.	Any `Stream` subclass (`FileStream`, `MemoryStream`) works wherever `Stream` is expected. A `Square : Rectangle` that changes the meaning of the width/height setters **violates** LSP.
I — Interface Segregation (ISP)	Clients should **not be forced to depend on members they never use** — prefer several small interfaces to one large one.	`IReadableRepo` / `IWritableRepo` instead of one fat `IRepository` with 20 methods. .NET's own `IEnumerable`, `IComparable`, `IDisposable` are small and focused.
D — Dependency Inversion (DIP)	**High-level modules should depend on abstractions**, not on concrete low-level classes.	A controller depends on `IMessageService` injected by the ASP.NET Core DI container — not on `new EmailService()`.

Small Code Illustration (SRP + DIP)

```
// BAD: one class does everything and creates its own dependency
public class OrderService {
    public void Place(Order o) {
        /* validate + save to SQL + send email + write log — 4 reasons to change */
        new SmtpClient().Send(...);
    }
}

// GOOD: one job each, and dependencies come in through abstractions
public class OrderService {
    private readonly IOrderRepository _repo;
    private readonly INotifier _notifier;
    public OrderService(IOrderRepository repo, INotifier notifier)   // DIP
        { _repo = repo; _notifier = notifier; }

    public void Place(Order o) { _repo.Save(o); _notifier.Notify(o); }   // SRP
}
// Switching e-mail to SMS = register a different INotifier. OrderService is untouched (OCP).
```

Importance in Maintainable and Extensible Software

1. **Maintainability:** small, single-purpose classes (SRP) **localise change** — fixing one behaviour cannot silently break an unrelated feature, which lowers long-term maintenance cost.
2. **Extensibility with low risk:** OCP lets new features arrive as **new classes or plug-ins** rather than edits to stable code, so tested functionality stays untouched and regressions are rare.
3. **Reliability and correct polymorphism:** LSP guarantees that inheritance hierarchies behave predictably, preventing subtle bugs when a derived type replaces a base type.
4. **Loose coupling and flexibility:** DIP and ISP decouple modules behind slim abstractions, so a database, logger or payment gateway can be swapped without a ripple of changes — essential in evolving enterprise systems.
5. **Testability:** dependencies expressed as interfaces are easily replaced by **mocks**, enabling fast, isolated unit tests — the backbone of TDD and reliable CI/CD pipelines.
6. **Readability and team productivity:** focused classes and interfaces are self-documenting, new team members understand the code faster, and parallel development is easier because modules meet only through contracts.
7. **Reusability:** cohesive, decoupled components can be reused across projects and layers.
8. **Architectural foundation:** SOLID underpins the patterns used in .NET — Repository, DI/IoC, middleware, Clean/Onion architecture and microservices — making systems scalable in both code size and team size.

> **Conclusion:** SOLID reduces **rigidity, fragility and immobility** in code. The result is software that is cheaper to change, safer to extend, easier to test and more resilient to changing requirements — exactly the goals of professional software engineering.

OR — Alternative for Q5 (b)

Q5 (b) OR. Explain the concept of state management in ASP.NET applications. Discuss the different client-side and server-side state management techniques with suitable examples.

8 Marks

> **Why state management is needed. HTTP is a stateless protocol** — each request is completely independent, and the web server forgets a page's controls, variables and user data as soon as the response has been sent. **State management** is the set of techniques ASP.NET provides to **preserve data** (user identity, shopping cart, form values, preferences) **across postbacks, requests and sessions**.

The techniques are divided into two families: **client-side** (the data is stored on, or travels with, the browser) and **server-side** (the data is stored in server memory or a backing store).

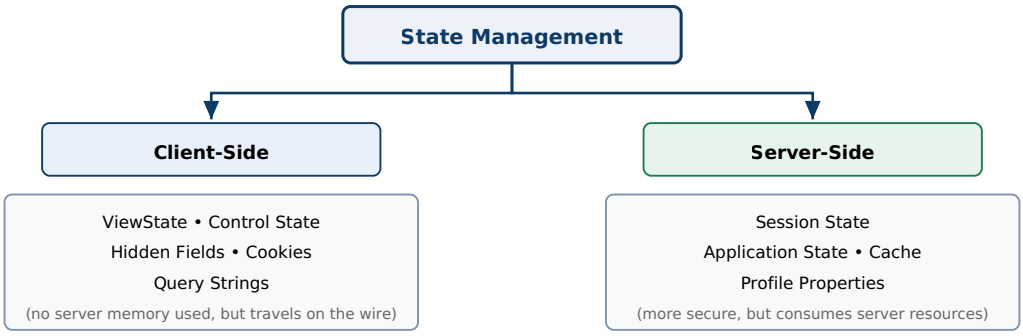

Fig 5.3: Classification of ASP.NET state management techniques

A. Client-Side Techniques

1. **ViewState** — ASP.NET automatically serialises the values of the page and its controls into a hidden, Base64-encoded field called `__VIEWSTATE`, and restores them on the next postback **of the same page**. It needs no server memory but **increases page size** because it travels with every postback.
   `ViewState["Count"] = 5; int c = (int)ViewState["Count"];`
2. **Control State** — a small, **non-disableable** portion of ViewState reserved for data a control absolutely needs in order to work (used mainly by control developers). It survives even when ViewState is turned off.
3. **Hidden Fields** — `<asp:HiddenField>` stores a small value inside the form. It is visible in "View Source", so it must **never** hold sensitive data.
4. **Cookies** — small text files (≈4 KB) stored by the browser and sent with every request to that site. They may be **session cookies** (deleted when the browser closes) or **persistent** (with an expiry date). Good for preferences and "remember me". Users can disable or edit them, so never store sensitive data unencrypted.
   `Response.Cookies["theme"].Value = "dark";`
5. **Query Strings** — data appended to the URL, e.g. `Details.aspx?id=5&page=2`, read with `Request.QueryString["id"]`. Simple and bookmarkable, but **visible, length-limited and easily tampered with**.

B. Server-Side Techniques

1. **Session State** — **per-user** data kept on the server and identified by a session ID cookie; available on all pages until it times out (20 minutes by default). Ideal for a shopping cart or logged-in user details, at the cost of server memory.
   `Session["Cart"] = cart;`
   Storage modes: *InProc* (fastest, in web-server memory, lost on restart), *StateServer* (separate Windows service), *SQLServer* (durable and web-farm safe), and in ASP.NET Core *distributed caches* such as Redis.
2. **Application State** — **global** data shared by **all users** of the application, e.g. `Application["Visitors"]`, usually initialised in `Global.asax`. It must be locked with `Application.Lock()` / `UnLock()` before updating, because many users may write at once. Lost when the application restarts, and not web-farm aware.
3. **Cache** — a high-performance, application-wide store with **expiration policies** (absolute or sliding), priorities and dependencies (e.g. invalidate when a file or table changes). It is used to avoid repeating expensive database queries. ASP.NET Core provides `IMemoryCache` and `IDistributedCache`.
4. **Profile Properties** — strongly typed, **persistent per-user** data stored automatically in a database through a provider; unlike Session it **survives** timeouts, browser closing and application restarts.

Comparison

Technique	Scope	Stored where	Lifetime	Best used for
ViewState	Single page	Client (hidden field)	Across postbacks	Control values on one page
Hidden field	Single page	Client (form)	Across postbacks	Small non-secret values
Cookies	Whole site	Client disk/ memory	Until expiry set	Preferences, "remember me"
Query string	Between pages	URL	One request	Small non-secret parameters, bookmarkable links
Session	One user, all pages	Server	Until timeout	Cart, logged-in user data
Application	All users	Server	Application lifetime	Global counters, shared settings
Cache	All users	Server	Policy based	Expensive query results
Profile	One user	Database	Permanent	Long-term user preferences

How to choose: use **client-side** state for small, non-sensitive, page-level data (it costs no server memory and scales easily); use **server-side** state for sensitive or larger per-user / application data (more secure, but consumes server resources).
In ASP.NET Core, ViewState and Application state no longer exist — their jobs are done by **TempData, Session, caching services (IMemoryCache / IDistributedCache)** and explicit model binding.

Q6 (a). Explain dependency injection in ASP.NET Core. Discuss its advantages and describe the commonly used service lifetimes. 7 Marks

Concept. Dependency Injection (DI) is a design pattern that implements the **Inversion of Control (IoC)** principle: instead of a class creating its own dependencies with `new`, the dependencies are **declared as interfaces** and are **supplied ("injected") from outside** by a container. ASP.NET Core has a **built-in IoC container** — services are registered in `Program.cs` and the container automatically creates and supplies them, most commonly through **constructor injection**.

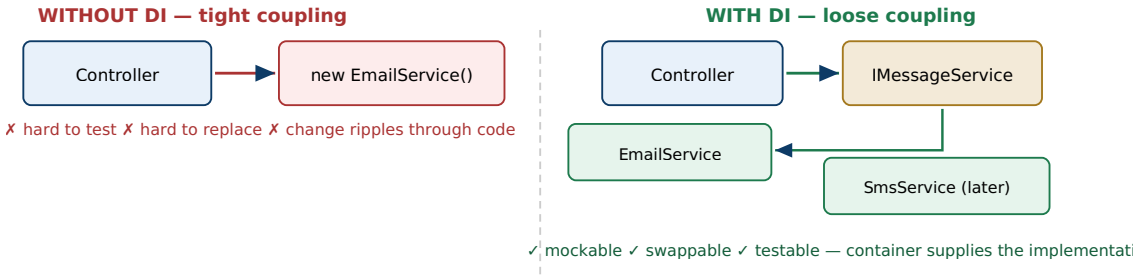

Fig 6.1: DI decouples the consumer from the concrete implementation

Implementation in ASP.NET Core (four steps)

```
// 1. Abstraction
public interface IMessageService { void Send(string to, string msg); }

// 2. Implementation
public class EmailService : IMessageService
{
    public void Send(string to, string msg) => Console.WriteLine($"Email to {to}: {msg}");
}

// 3. Registration in Program.cs — the "composition root"
builder.Services.AddScoped<IMessageService, EmailService>();

// 4. Constructor injection — the container supplies EmailService automatically
public class NotifyController : Controller
{
    private readonly IMessageService _svc;
    public NotifyController(IMessageService svc) { _svc = svc; }

    public IActionResult Index()
    {
        _svc.Send("arpan@ncit.edu.np", "Result published");
        return View();
    }
}
```

To switch later from e-mail to SMS, **only the registration line changes** (`AddScoped<IMessageService, SmsService>()`) — the controller code is not touched at all.

Service Lifetimes

Lifetime	Registration	When an instance is created	Typical use
Transient	`AddTransient<I,T>()`	**Every time** the service is requested — a brand-new object each time.	Lightweight, stateless services (validators, small helpers)
Scoped	`AddScoped<I,T>()`	**Once per HTTP request** — the same object is shared everywhere inside that request.	`DbContext`, unit-of-work, repositories
Singleton	`AddSingleton<I,T>()`	**Once for the whole application** — the same object for every request and every user.	Caches, configuration, loggers, in-memory lookup data

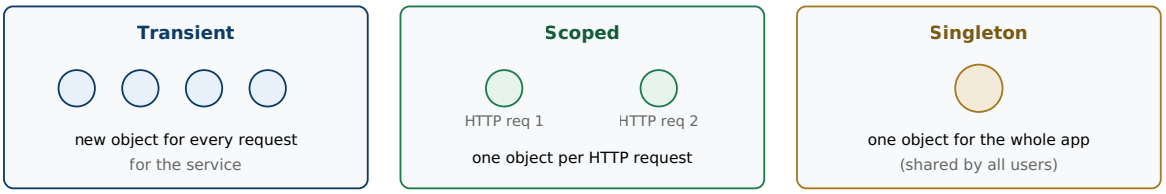

Fig 6.2: The three service lifetimes

> **Important rule (captive dependency):** never inject a **Scoped** service into a **Singleton**. The singleton would hold on to that object forever — for example a `DbContext` captured by a singleton becomes shared across requests and is not thread-safe. ASP.NET Core detects and reports this error at startup in the development environment.

Advantages of Dependency Injection

1. **Loose coupling:** classes depend on **abstractions**, not concrete types — a direct application of the Dependency Inversion Principle of SOLID.
2. **Testability:** real services can be replaced by **mocks or stubs** in unit tests (e.g. a fake `IMessageService`), so tests need no database, network or e-mail server.
3. **Maintainability and flexibility:** an implementation can be replaced or upgraded in **one place** without modifying any consumer.
4. **Centralised lifetime management:** the container creates and correctly **disposes** objects (for example, exactly one `DbContext` per request).
5. **Cleaner, reusable code:** no `new` statements scattered everywhere; cross-cutting services such as logging, configuration and caching are shared consistently.
6. **Built into the framework:** ASP.NET Core itself supplies `ILogger`, `IConfiguration`, `IHttpClientFactory` and `DbContext` through the same container, so the pattern is followed naturally.

Q6 (b). Explain the development of RESTful Web APIs using ASP.NET Core. Discuss HTTP methods, endpoints, request handling, response handling and status codes. 8 Marks

Definition. A **Web API** is an HTTP-based service that exposes application data and functionality over the web so that many kinds of clients — browsers, mobile apps, desktop apps, IoT devices and other servers — can consume it. A **RESTful** Web API follows the REST style: **resources** are identified by URLs, standard **HTTP verbs** act on them, communication is **stateless**, and data is normally exchanged as **JSON**. In ASP.NET Core these are built with controllers deriving from `ControllerBase` (or with Minimal APIs).

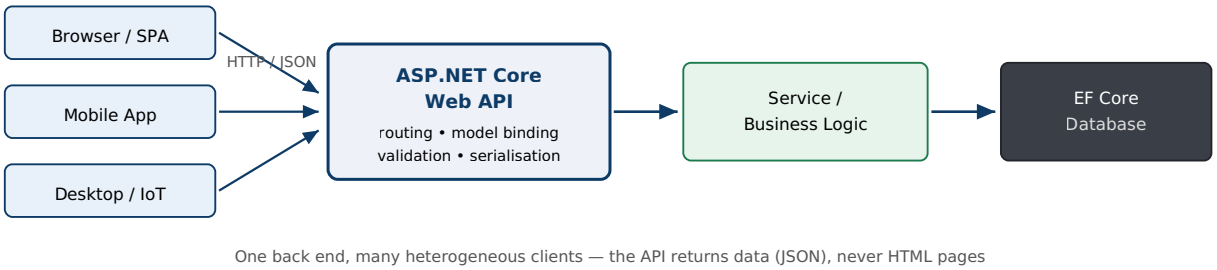

Fig 6.3: Multiple clients consuming a single ASP.NET Core Web API

1. Creating the API and its Endpoints

An **endpoint** is the combination of a **URL (route)** and an **HTTP verb** that identifies one operation on a resource. ASP.NET Core uses **attribute routing**; the `[ApiController]` attribute switches on automatic model validation, binding-source inference and problem-details error responses.

```
[ApiController]
[Route("api/[controller]")]                 // base route → api/students
public class StudentsController : ControllerBase
{
    private readonly SchoolContext _db;
    public StudentsController(SchoolContext db) => _db = db;   // DI

    [HttpGet]                               // GET  api/students
    public async Task<ActionResult<IEnumerable<Student>>> GetAll()
        => await _db.Students.ToListAsync();                       // 200 OK

    [HttpGet("{id}")]                       // GET  api/students/5
    public async Task<ActionResult<Student>> Get(int id)
    {
        var s = await _db.Students.FindAsync(id);
        return s is null ? NotFound() : Ok(s);                     // 404 or 200
    }

    [HttpPost]                              // POST api/students
    public async Task<IActionResult> Create([FromBody] Student s)
    {
        if (!ModelState.IsValid) return BadRequest(ModelState);     // 400
        _db.Students.Add(s);
        await _db.SaveChangesAsync();
        return CreatedAtAction(nameof(Get), new { id = s.Id }, s);  // 201 + Location
    }

    [HttpPut("{id}")]                       // PUT  api/students/5
    public async Task<IActionResult> Update(int id, Student s)
    {
        if (id != s.Id) return BadRequest();
        _db.Entry(s).State = EntityState.Modified;
        await _db.SaveChangesAsync();
        return NoContent();                                         // 204
    }

    [HttpDelete("{id}")]                    // DELETE api/students/5
    public async Task<IActionResult> Delete(int id)
    {
        var s = await _db.Students.FindAsync(id);
        if (s is null) return NotFound();
        _db.Students.Remove(s);
        await _db.SaveChangesAsync();
        return NoContent();                                         // 204
    }
}
```

2. HTTP Methods (verbs) and Endpoint Design

Verb	Typical endpoint	Meaning	Safe / Idempotent	Success status
GET	`/api/students`, `/api/students/5`	Read a collection or one resource	Safe + idempotent	200 OK
POST	`/api/students`	Create a new resource	Neither	201 Created
PUT	`/api/students/5`	Replace an existing resource completely	Idempotent	204 No Content
PATCH	`/api/students/5`	Update part of a resource	Not guaranteed	200 / 204
DELETE	`/api/students/5`	Remove a resource	Idempotent	204 No Content

Endpoint naming rules: use **plural nouns** for resources (`/api/students`), never verbs (`/api/getStudent` is wrong — the verb is already in the HTTP method); nest for relationships (`/api/students/5/courses`); use query strings for filtering and paging (`?page=2&sort=gpa`); and version the API (`/api/v1/students`).

3. Request Handling

- **Routing** matches the incoming URL and verb to exactly one action method.
- **Model binding** fills the action parameters automatically from the request: `[FromRoute]` (URL segment), `[FromQuery]` (query string), `[FromBody]` (JSON body), `[FromHeader]` and `[FromForm]`.
- **Model validation** runs from data annotations (`[Required]`, `[Range]`, `[EmailAddress]`). With `[ApiController]`, an invalid model automatically returns **400 Bad Request** with a problem-details body — no manual check needed.
- **Content negotiation** reads the `Accept` header and formats the answer as JSON (default) or XML if that formatter is added.
- **Middleware** handles cross-cutting concerns before the controller runs: exception handling, HTTPS redirection, CORS, authentication and authorization.

4. Response Handling

- Return the object directly, or an `ActionResult<T>` using helper methods: `Ok(data)`, `CreatedAtAction(...)`, `NoContent()`, `NotFound()`, `BadRequest(...)`, `Unauthorized()`, `Forbid()`.
- The object is **serialised to JSON** automatically by `System.Text.Json` and the correct `Content-Type` header is set.
- **DTOs** (Data Transfer Objects) should be returned instead of raw entities, so that internal fields and password hashes are never exposed.
- Errors are returned in the standard **ProblemDetails** format (RFC 7807), and a global exception-handling middleware converts unhandled exceptions into a clean **500** response instead of a stack trace.
- **Swagger / OpenAPI** documents every endpoint and lets clients test the API from a browser.

5. HTTP Status Codes

Class	Code	Meaning and when the API returns it
2xx Success	200 OK	Request succeeded and data is returned (GET, PUT with body)
	201 Created	New resource created — includes a `Location` header pointing at it (POST)
	204 No Content	Succeeded but there is nothing to return (PUT, DELETE)
3xx Redirect	301 / 304 Not Modified	Resource moved, or the client's cached copy is still valid
4xx Client error	400 Bad Request	Malformed request or failed validation
	401 Unauthorized	No valid credentials/token supplied — *authentication* failed
	403 Forbidden	Identity known but not permitted — *authorization* failed
	404 Not Found	The requested resource does not exist
	409 Conflict / 422	Conflicting update (concurrency) / semantically invalid entity
5xx Server error	500 Internal Server Error	Unhandled exception on the server
	503 Service Unavailable	Server overloaded or down for maintenance

Advantages of ASP.NET Core Web API

1. **Client independence** — one back end serves web, Android, iOS and IoT clients.
2. **Lightweight and fast** — plain HTTP + JSON instead of heavy SOAP envelopes; Kestrel is among the fastest web servers.
3. **Full use of HTTP** — caching, content negotiation, status codes, headers and versioning.
4. **Statelessness → scalability** — requests can be load-balanced across many servers, which suits microservices.
5. **Framework features built in** — routing, model binding, validation, filters, dependency injection and security (JWT / OAuth).
6. **Testability and tooling** — endpoints can be exercised with Postman or Swagger and unit-tested without any UI.

OR — Alternative for Q6 (b)

Q6 (b) OR. Discuss the benefits and challenges of adopting a microservices approach in enterprise .NET applications.

8 Marks

> **Concept.** A **microservices architecture** breaks an application into a set of **small, autonomous services**, each built around **one business capability** (Orders, Payments, Inventory), each running in **its own process**, owning **its own database**, and communicating over lightweight protocols — HTTP/REST, gRPC, or asynchronous messaging (RabbitMQ, Azure Service Bus). This is the opposite of a **monolith**, where every module is compiled and deployed as one single unit.

In the .NET world, microservices are typically ASP.NET Core Web API or gRPC services, packaged with **Docker** and orchestrated by **Kubernetes** or Azure Container Apps, with **Dapr** or **.NET Aspire** helping with service discovery, configuration and observability.

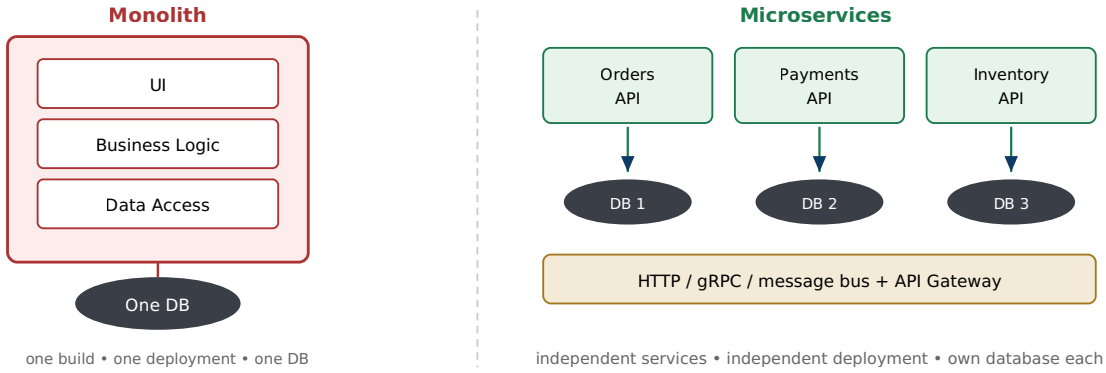

Fig 6.4: Monolith versus microservices

Benefits

1. **Independent deployment and agility:** each service is built, tested and released on its own — small, frequent, low-risk deployments through CI/CD, instead of redeploying the entire system for a one-line change.
2. **Selective scalability:** only the busy services are scaled out (e.g. Orders during a festival sale), which is far cheaper than cloning a whole monolith just to handle one hot module.
3. **Fault isolation and resilience:** a failure in one service does not take down the whole application, provided resilience patterns are used — **circuit breakers, retries and timeouts** (Polly library in .NET).
4. **Technology and data-store freedom:** each team can pick the best tool for its service — different .NET versions or even other languages, and **polyglot persistence** (SQL Server, PostgreSQL, Redis, Cosmos DB).
5. **Team autonomy and clear ownership:** small teams own a service end-to-end, aligned with business domains (Domain-Driven Design **bounded contexts**), which reduces coordination overhead in a large enterprise.
6. **Easier to understand and replace:** a small code base can be learned quickly, rewritten or retired — something impossible with a million-line monolith.

Challenges

1. **Distributed-system complexity:** in-process method calls become network calls, so **latency, partial failures, retries and timeouts** must be handled everywhere, and debugging spans many services.
2. **Data consistency:** each service owns its own database, so a normal ACID transaction across services is impossible. Teams must adopt **eventual consistency, the Saga pattern** and event-driven designs — much harder than a single `SaveChanges()`.
3. **Operational overhead (DevOps maturity required):** dozens of services need containerisation, orchestration (Kubernetes), service discovery, an **API gateway** (Ocelot / YARP), configuration and secrets management.
4. **Observability:** centralised logging, **distributed tracing** (OpenTelemetry, Application Insights), health checks and dashboards become mandatory just to know what the system is doing.
5. **Testing difficulty:** integration and end-to-end testing across independently deployed, independently versioned services is far harder than testing one process; contract testing becomes necessary.
6. **Network and security surface:** every inter-service call must be secured (mTLS, tokens) and adds serialisation and bandwidth cost.
7. **Organisational cost:** it demands skilled teams and a cultural change. For a small application the overhead outweighs the benefits.

> **Conclusion:** microservices give an enterprise **agility, selective scalability and resilience**, but the price is **distributed complexity, consistency problems and heavy DevOps investment**. The practical advice is "**start with a well-structured modular monolith**" and split out microservices only when the domain size, team structure and scaling needs genuinely justify the cost.

Q7 (a). Short Note: Garbage Collection mechanism in .NET — 5 Marks

> **Definition. Garbage Collection (GC)** is the CLR's **automatic memory management** service. Reference-type objects are allocated on the **managed heap**; when an object can no longer be reached by any live reference, the GC reclaims its memory automatically. Developers therefore never call `free()` or `delete`, which eliminates memory leaks, double-frees and dangling pointers.

Phases of a Collection

1. **Marking** — starting from the **application roots** (static fields, local variables of active threads, CPU registers, GC handles), the GC walks the object graph and **marks every reachable object as live**. Anything not marked is garbage.
2. **Relocating (planning)** — references to the surviving objects are updated to point at their new addresses.
3. **Sweeping and compacting** — the memory of the dead objects is reclaimed and the survivors are **moved together**, so free memory forms one contiguous block. This removes fragmentation and makes the next allocation extremely fast — just moving a pointer forward.

Generational Model

The GC is **generational**, based on the observation that **most objects die young**. Collecting only the young generation keeps pauses very short.

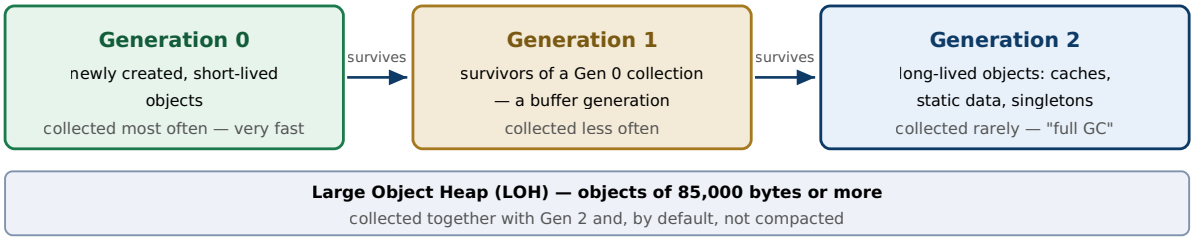

Fig 7.1: The generational managed heap in .NET

When Does the GC Run?

- Generation 0 has used up its **allocation budget** (the most common trigger).
- The operating system reports **low physical memory**.
- `GC.Collect()` is called explicitly — **discouraged** in normal application code.

GC Flavours and Related Features

- **Workstation vs Server GC:** Workstation GC optimises responsiveness for client applications; **Server GC** uses one heap and one GC thread **per CPU core** for high throughput and is the default for ASP.NET Core.
- **Background (concurrent) GC:** expensive Gen 2 collections run mostly in the background while application threads keep running, which keeps pauses short.
- **Finalization and IDisposable:** the GC frees only **managed memory**. **Unmanaged** resources (file handles, database connections, sockets) must be released deterministically through `IDisposable.Dispose()` / the `using` statement. A finalizer (`~ClassName`) is only a safety net and actually **delays** reclamation by one extra GC cycle, so the recommended pattern is `Dispose()` + `GC.SuppressFinalize(this)`.

- **Useful API:** `GC.Collect()`, `GC.GetGeneration(obj)`, `GC.GetTotalMemory()`, `GC.WaitForPendingFinalizers()`.

Advantages

1. Removes whole classes of memory bugs — leaks, dangling pointers, double frees — improving reliability and security.
2. Very fast "pointer-bump" allocation and automatic defragmentation through compaction.
3. The generational + background design keeps pause times low while maintaining throughput.
4. Developers concentrate on business logic instead of memory bookkeeping.

Q7 (b). Short Note: Authentication and Authorization — 5 Marks

Authentication answers the question **"Who are you?"** — it verifies a user's identity from credentials, a token or a certificate.
Authorization answers the question **"What are you allowed to do?"** — it decides whether the already-identified user may access a particular resource.
Authentication always happens first. In the ASP.NET Core pipeline this order is enforced by placing `UseAuthentication()` before `UseAuthorization()`.

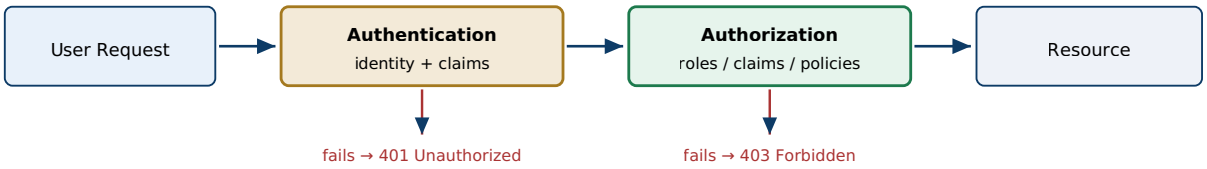

Fig 7.2: Security pipeline — authenticate first, then authorize

Authentication Mechanisms in ASP.NET

1. **Forms Authentication** (classic ASP.NET) — an unauthenticated user is redirected to a login page; on success an **encrypted authentication ticket** is stored in a cookie and checked on every request. Configured in `web.config` with `<authentication mode="Forms">`.
2. **Windows Authentication** — uses the user's Windows / Active Directory account through NTLM or Kerberos. Perfect for intranet applications; no login page is needed.
3. **ASP.NET Core Identity** — a complete membership system: user and role stores in a database through EF Core, password hashing, account lockout, two-factor authentication and external logins.
4. **Cookie Authentication** (ASP.NET Core) — after login a **signed cookie** carries the user's `ClaimsPrincipal` on each request; the default choice for browser applications.
5. **JWT Bearer Token Authentication** — for Web APIs, SPAs and mobile apps. The server issues a signed **JSON Web Token**, and the client sends it in the `Authorization: Bearer <token>` header. Being **stateless**, it scales well and suits microservices.
6. **External providers (OAuth 2.0 / OpenID Connect)** — sign in with Google, Facebook, Microsoft or Azure AD; the modern replacement for the legacy Passport service.

Authorization Mechanisms

1. **Simple authorization** — `[Authorize]` on a controller or action requires **any** authenticated user; `[AllowAnonymous]` opts a specific action out.
2. **Role-based** — access depends on membership of a role: `[Authorize(Roles = "Admin,Manager")]`. In classic ASP.NET the equivalent is URL authorization: `<allow roles="Admin"/> <deny users="*"/>`.
3. **Claims-based** — decisions use **claims**, which are key–value facts stored in the identity (e.g. `Department = "IT"`, `Age = 21`).
4. **Policy-based** (ASP.NET Core) — named policies combine several requirements with custom handlers and are the recommended modern approach.
5. **Resource-based** — an imperative check against a particular object (e.g. only the owner of a document may edit it) using `IAuthorizationService`.

```
builder.Services.AddAuthorization(o =>
    o.AddPolicy("Adult", p => p.RequireClaim("Age")
        .RequireAssertion(ctx => int.Parse(ctx.User.FindFirst("Age")!.Value) >= 18)));

// pipeline order matters
app.UseAuthentication();     // who are you?
app.UseAuthorization();      // what may you do?

[Authorize(Policy = "Adult")]
public IActionResult Restricted() => View();
```

Summary: authentication **establishes identity** (Forms, Windows, Identity, cookies, JWT/OAuth), and authorization then **grants or denies access** using roles, claims or policies. Together they form the layered security model of ASP.NET. Failure of the first returns **401**; failure of the second returns **403**.

Q7 (c). Short Note: Abstract Class vs Interface

5 Marks

Both **abstract classes** and **interfaces** are used to achieve **abstraction and polymorphism** in .NET, and neither can be instantiated directly. They differ in purpose: an **abstract class represents an "is-a" relationship together with shared implementation**, whereas an **interface represents a "can-do" contract of capabilities**.

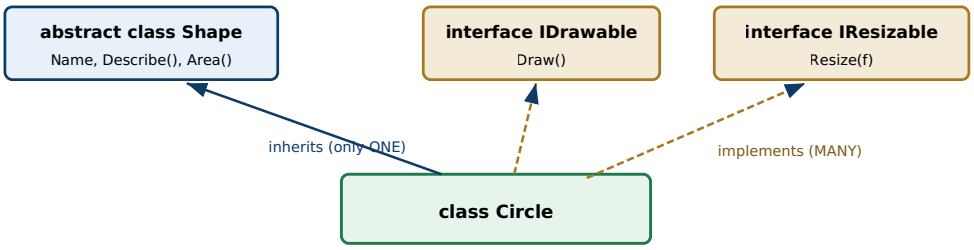

Fig 7.3: One base class may be inherited, but many interfaces may be implemented

Comparison

Basis	Abstract Class	Interface
Keyword	`abstract class`	`interface`
Implementation	Can contain both abstract members and fully implemented (concrete) methods, fields and constructors	Traditionally only declarations; C# 8+ allows **default implementations**, but still no instance fields or constructors
Inheritance	A class can inherit **only one** abstract class (single inheritance)	A class can implement **many** interfaces
Members allowed	Fields, properties, methods, events, constructors, destructors; any access modifier	Methods, properties, events, indexers; members are **public by default**; no instance fields
State	Can hold state (instance variables)	Cannot hold instance state
Instantiation	Cannot be instantiated directly	Cannot be instantiated directly
Versioning	Adding a new concrete method does **not** break derived classes	Adding a member **breaks all implementers** (unless a default implementation is provided)
Speed	Slightly faster member access	Small overhead due to interface dispatch
Relationship	"**is-a**" — a Dog *is an* Animal	"**can-do**" — a Dog *can be* IComparable

Code Illustration

```
public abstract class Shape                       // shared code + contract
{
    public string Name { get; set; }              // state is allowed
    public abstract double Area();                // must be overridden
    public void Describe() =>                     // concrete shared behaviour
        Console.WriteLine($"{Name} area = {Area()}");
}

public interface IDrawable  { void Draw(); }              // pure capability
public interface IResizable { void Resize(double f); }

public class Circle : Shape, IDrawable, IResizable        // 1 class + many interfaces
{
    public double Radius { get; set; }
    public override double Area() => Math.PI * Radius * Radius;
    public void Draw()   => Console.WriteLine("Drawing circle");
    public void Resize(double f) => Radius *= f;
}
```

When to Prefer Which

Prefer an Abstract Class when…	Prefer an Interface when…
Several **closely related** classes must share common code and state (e.g. `Shape` with `Name` and `Describe()`).	**Unrelated** classes need a common capability — `IComparable`, `IDisposable`, `IEnumerable` are implemented by hundreds of unrelated types.
You need **constructors, fields** or non-public members in the base type.	A class needs **multiple** behaviour contracts — C# forbids multiple base classes but allows many interfaces.
The base type will **evolve** — new concrete methods can be added later without breaking subclasses (e.g. `ControllerBase`, `DbConnection`).	You are doing **dependency injection and unit testing** — services are registered and mocked through interfaces (`IRepository`, `ILogger`) to keep coupling loose.
A default behaviour should exist that most subclasses can simply reuse.	You are defining a **plug-in or public API contract** where implementers must not inherit any implementation.

Conclusion: use an **abstract class** to **share implementation** among a family of related types; use an **interface** to **define a contract** that many unrelated types can fulfil and to keep components loosely coupled. In modern .NET the two are often combined — an interface for the contract and an abstract base class that provides a convenient default implementation of it.

— End of Model Answers —

17 answers • all OR alternatives included • 2026 question paper
Nepal College of Information Technology | .NET Technologies (Elective)

Prepared by **Arpan Adhikari**

Last-minute revision order (highest value first):
1. Diagrams — architecture, page life cycle, middleware/DI, GC generations, monolith vs microservices
2. Tables — Framework vs Core vs .NET, lifetimes, status codes, abstract vs interface
3. Definitions in the shaded boxes — write these first in every answer
4. Code snippets — 6–10 lines is enough to earn the marks